

Notas de Clase

Álgebra Lineal Aplicada

Christian Bustos Betancur

Febrero 2026

Índice general

1. Introducción al Álgebra Lineal en Julia	3
1.1. Principales Problemas del Álgebra Lineal Numérica	7
1.2. Valores y Tipos en Julia	8
1.2.1. Categorías generales de tipos	9
1.2.2. Tipos numéricos	10
1.2.3. Variables en Julia	11
1.3. Vectores y Matrices	12
1.3.1. Vectores	12
1.3.2. Matrices	13
1.3.3. Matriz fila (Row vector) y matriz columna (Column vector)	13
1.3.4. Matriz cero (Zero matrix)	14
1.3.5. Matriz identidad (Identity matrix)	15
1.4. Operaciones con Vectores y Matrices	16
1.4.1. Operaciones con Vectores	17
1.4.1.1. Suma de vectores	17
1.4.1.2. Producto por escalar	18
1.4.1.3. Producto punto (Dot product)	19
1.4.2. Operaciones con Matrices	20
1.4.2.1. Suma de matrices	20
1.4.2.2. Producto matriz-escalar	20
1.4.2.3. Producto matricial	21
1.4.3. Operadores en Julia	21
1.4.3.1. Operadores Aritméticos	21
1.4.3.2. Operadores de Comparación	22
1.4.3.3. Operadores Lógicos	22
2. Librería LinearAlgebra	23
2.1. Sistema de ayuda en Julia	24
2.2. Operadores para Álgebra Lineal	25
2.2.1. Norma de un vector	26
2.2.2. Producto cruz	27
2.2.3. Norma de una matriz	28
2.2.4. Transpuesta de una matriz	29
2.2.5. Inversa de una matriz	29
2.3. Consideraciones prácticas y ejemplos	30

3. Error Numérico y Problemas Mal Condicionados	38
3.1. Error de representación en punto flotante	38
3.2. Error de redondeo	40
3.3. Propagación del error	41
3.4. Error de cancelación numérica	41
3.5. Problemas mal condicionados	42
4. Transformaciones Lineales y Afines	48
4.1. Transformaciones afines en 2D y 3D	49
4.2. Isometrías	51
4.2.1. Traslaciones	51
4.2.2. Rotaciones	51
4.2.3. Reflexiones	51
4.3. Imágenes y figuras en Julia	52
4.4. Gráficos Vectoriales	54
4.5. Interpolación	67
4.5.1. Interpolación mediante la matriz de Vandermonde	68
4.5.2. Interpolación de Lagrange	69
4.5.3. Interpolación a trozos	70
5. Transformaciones y Matrices	75
5.1. Transformaciones lineales	75
5.1.1. Representación mediante matrices	76
5.1.2. Composición de transformaciones	76
5.1.3. Transformación inversa	76
5.2. Matrices Invertibles	77
5.2.1. Definición y caracterizaciones equivalentes	77
5.2.2. Teorema de la matriz invertible	77
5.2.3. Relación con sistemas lineales	77
5.2.4. Inversa de matrices triangulares	78
5.2.5. Inversa de matrices ortogonales	78
Bibliografía	82

Capítulo 1

Introducción al Álgebra Lineal en Julia

Motivación y objetivos

En el presente documento se presenta una introducción del álgebra lineal numérica utilizando el lenguaje de programación Julia.

Julia es un lenguaje de programación de alto nivel para el cálculo científico y numérico, diseñado para ofrecer alto rendimiento [1].

Una de sus principales ventajas es que ofrece la facilidad de lenguajes como **Python** o **MATLAB** con un rendimiento comparable a **C** y **Fortran**.

Por estas razones, aprovecharemos todas las capacidades que nos brinda Julia para el desarrollo, la implementación y la resolución de problemas comunes del álgebra lineal numérica, para la explicación de las funciones del lenguaje y su uso en la práctica se tomará como referencia la documentación oficial de Julia [2], además utilizaremos **Google Colab**, una plataforma en la nube la cual nos permite ejecutar código desde el navegador.

Configuración del entorno en Google Colab

Para comenzar a programar en Julia utilizaremos **Google Colab**, una plataforma en la nube que permite ejecutar código desde el navegador.

El proceso de configuración es el siguiente:

1. Crear un nuevo cuaderno en Google Colab.

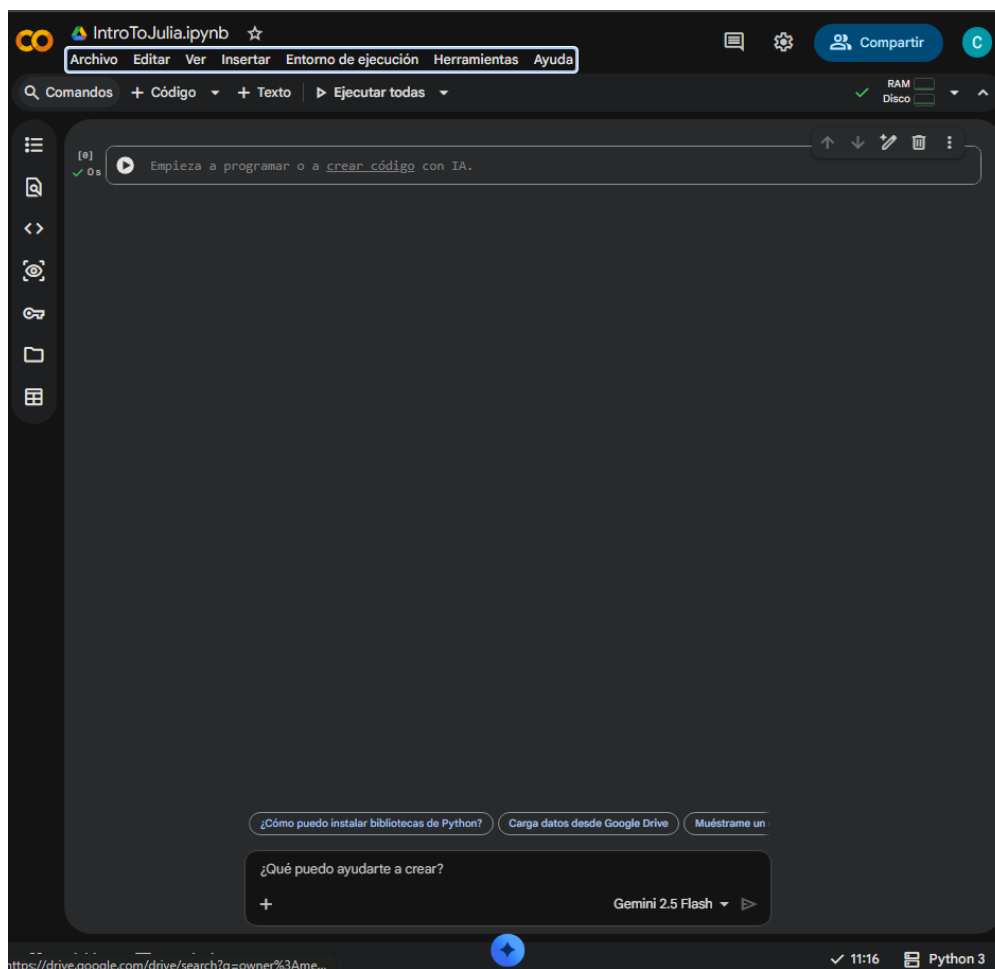


Figura 1.1: Creación de un nuevo cuaderno en Google Colab

2. Acceder a *Entorno de ejecución* y seleccionar *Cambiar tipo de entorno de ejecución*.

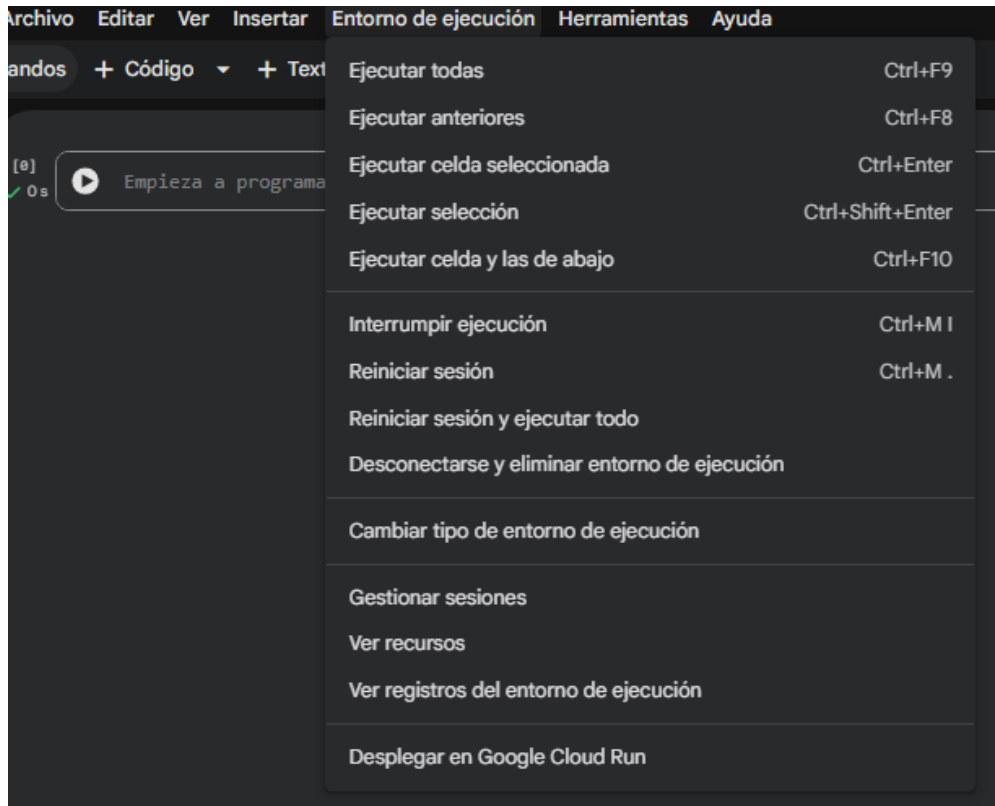


Figura 1.2: Configuración del entorno de ejecución

3. Cambiar el lenguaje de *Python* a *Julia* y presionar *Guardar*.

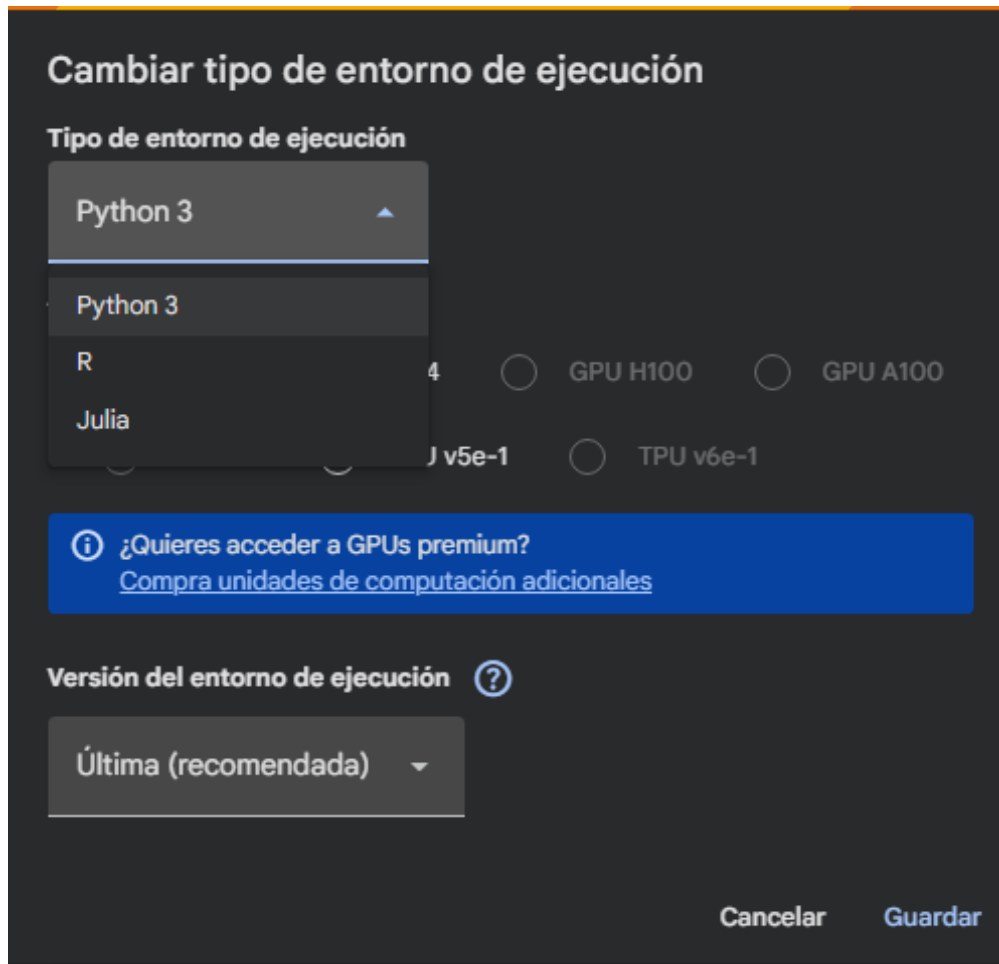


Figura 1.3: Selección de Julia como entorno de ejecución

Una vez realizados estos pasos, el entorno estará listo para comenzar a programar.

1.1. Principales Problemas del Álgebra Lineal Numérica

La organización presentada sigue la estructura del álgebra lineal numérica descrita por Trefethen y Bau [3] y Golub y Van Loan [4].

Trabajaremos principalmente en el espacio $\mathbb{R}^n$, aunque muchas formulaciones se presentan en $\mathbb{C}^n$.

Sistemas Lineales

Resolver el sistema lineal

$$Ax = b$$

donde $A \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$ y $b \in \mathbb{R}^m$.

Problema de Mínimos Cuadrados

Sea $A \in \mathbb{R}^{m \times n}$ y $b \in \mathbb{R}^m$. Encontrar $x \in \mathbb{R}^n$ que minimiza:

$$\|Ax - b\|_2$$

De manera equivalente:

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2$$

Problema de Autovalores

Sea $A \in \mathbb{R}^{n \times n}$. Encontrar $\lambda \in \mathbb{R}$ y $v \in \mathbb{R}^n \setminus \{0\}$ tales que

$$Av = \lambda v$$

Descomposición en Valores Singulares

Factorizar $A \in \mathbb{R}^{m \times n}$ como:

$$A = U\Sigma V^T$$

donde $U \in \mathbb{R}^{m \times m}$ y $V \in \mathbb{R}^{n \times n}$ son matrices ortogonales, y $\Sigma \in \mathbb{R}^{m \times n}$ es diagonal con entradas no negativas.

1.2. Valores y Tipos en Julia

El sistema de tipos del lenguaje Julia se describe detalladamente en la documentación oficial del lenguaje [2]. En esta sección se presentan los conceptos fundamentales necesarios para el cálculo científico y el álgebra lineal numérica.

En Julia, todos los datos que aparecen en un programa se representan como **valores**. Un valor es una entidad concreta almacenada en memoria, como un número, una cadena de texto o un arreglo.

El lenguaje presenta varias características importantes respecto a su sistema de tipos:

- No existe diferencia entre valores de objeto y valores primitivos: Todos los valores en Julia son objetos y pertenecen a algún tipo dentro del sistema de tipos del lenguaje.
- No existe un tipo fijo en tiempo de compilación. El único tipo que tiene un valor es su tipo real durante la ejecución del programa.
- Solo los valores, y no las variables, tienen tipos. Las variables actúan simplemente como nombres asociados a valores almacenados en memoria.

En Julia es posible consultar el tipo de un valor mediante la función `typeof`.

```
typeof(5)
typeof(3.14)
typeof(true)
```

```
-----
Int64
Float64
Bool
```

1.2.1. Categorías generales de tipos

- **Tipos abstractos.** No representan valores directamente, sino que definen una categoría de tipos relacionados dentro de la jerarquía del lenguaje. Por ejemplo, `Number` es un tipo abstracto que agrupa diferentes clases de números.
- **Tipos primitivos.** Son tipos definidos directamente a nivel de máquina y representan estructuras de datos básicas. Ejemplos importantes son `Int64` y `Float64`.
- **Tipos compuestos.** Permiten agrupar varios valores dentro de una misma estructura. Son similares a las estructuras (*struct*) en otros lenguajes y se utilizan para construir objetos más complejos.
- **Tipos paramétricos.** Son tipos que dependen de uno o más parámetros de tipo. Esto permite definir estructuras genéricas como

`Vector{Float64},     Matrix{Float64}`

donde el parámetro indica el tipo de los elementos contenidos.

Los tipos en Julia se organizan en una jerarquía. Los tipos abstractos definen categorías generales, mientras que los tipos concretos representan los valores que pueden almacenarse en memoria.

En el contexto del cálculo científico y el álgebra lineal numérica, los tipos más utilizados son los tipos numéricos y las estructuras paramétricas que representan vectores y matrices.

1.2.2. Tipos numéricos

Julia proporciona distintos tipos numéricos que permiten representar diferentes clases de números utilizados en computación científica.

- **Enteros:** `Int8`, `Int16`, `Int32`, `Int64`. Representan números enteros con distinta precisión. En arquitecturas de 64 bits, el tipo entero por defecto es `Int64`.
- **Enteros sin signo:** `UInt8`, `UInt16`, `UInt32`, `UInt64`. Permiten representar valores enteros no negativos.
- **Punto flotante:** `Float32`, `Float64`. Se utilizan para representar números reales mediante el estándar IEEE de coma flotante. El tipo por defecto es `Float64`.
- **Complejos:** `Complex{T}`. Representan números de la forma $a + bi$, donde el parámetro `T` indica el tipo de las partes real e imaginaria (por ejemplo, `Complex{Float64}`).
- **Racionales:** `Rational{T}`. Representan fracciones exactas entre enteros del tipo `T`.
- **Booleanos:** `Bool`. Admiten los valores lógicos `true` y `false`.

Estas estructuras se implementan en Julia mediante arreglos paramétricos como `Vector{T}` y `Matrix{T}`, donde el parámetro `T` indica el tipo de los elementos almacenados.

Estas estructuras serán utilizadas a lo largo de estas notas para representar objetos del álgebra lineal numérica.

1.2.3. Variables en Julia

Una variable en Julia es un nombre asociado a un valor.

Se utiliza para almacenar un resultado o dato con el fin de reutilizarlo posteriormente en el programa.

Para definir una variable lo podemos hacer escribiendo inicialmente el nombre de la variable seguido del operador = y le asignamos un valor.

```
nombre_variable = valor
```

Ejemplo:

```
x = 5
typeof(x)
-----
Int64
```

En este caso, la variable `x` queda asociada al valor entero 5, cuyo tipo es `Int64`.

Buenas prácticas de nomenclatura

- Utilizar nombres descriptivos (`edad`, `matrizA`).
- Evitar espacios en los identificadores.
- No comenzar los nombres con números.
- Mantener consistencia en el estilo (por ejemplo, `snake_case`).

1.3. Vectores y Matrices

La presentación de vectores, matrices y las operaciones básicas del álgebra lineal que se utilizan en estas notas sigue la exposición desarrollada en el texto *Introduction to Applied Linear Algebra* de Boyd y Vandenberghe [5].

En Julia, los vectores y matrices se representan mediante arreglos (*arrays*). Estas estructuras permiten almacenar colecciones ordenadas de valores numéricos.

1.3.1. Vectores

Un vector $\mathbf{v} \in \mathbb{R}^n$ es una lista ordenada con una cantidad finita de elementos, donde cada elemento o entrada $\mathbf{v}_i$ corresponde a una componente del vector.

Podemos representar los vectores como una lista vertical (vector columna) u horizontal (vector fila) rodeada por corchetes o paréntesis:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} = \begin{pmatrix} v_1 \\ \vdots \\ v_n \end{pmatrix} = [v_1, \dots, v_n] = (v_1, \dots, v_n)$$

El tamaño del vector $\mathbf{v}$ viene dado por la cantidad de elementos que contiene.

Un vector con n elementos se denomina un n -vector. En este caso, la dimensión del vector puede representarse como $n \times 1$ si se escribe como vector columna, o $1 \times n$ si se escribe como vector fila.

Representacion de un vector en Julia

En Julia, un vector se representa mediante un arreglo (*array*) unidimensional. Estos se definen utilizando corchetes `[]`, donde los elementos pueden separarse mediante comas `,` o punto y coma `;`.

Por ejemplo:

```
v = [1, 2, 3]
```

o equivalentemente

```
v = [1; 2; 3]
```

1.3.2. Matrices

Podemos definir una matriz $\mathbf{A} \in \mathbb{R}^{m \times n}$ como un arreglo rectangular de números organizados en filas y columnas, donde m es el número de filas y n el número de columnas. Cada a_{ij} es una entrada(o elemento) de la matriz ubicada en la fila i y la columna j

```
A = [1 2; 3 4] #Definimos nuestra matriz
```

```
-----  
2x2 Matrix{Int64}:
```

```
 1  2  
 3  4
```

1.3.3. Matriz fila (Row vector) y matriz columna (Column vector)

Una matriz fila es una matriz de dimensión $1 \times n$.

Una matriz columna es una matriz de dimensión $n \times 1$.

```
row = [1 2 3]      # (1x3) Matrix{Int64}  
col = [1; 2; 3]    # (3x1) 3-element Vector{Int64}  
row, col
```

```
-----  
([1 2 3], [1, 2, 3])
```

Nota:

En Julia, las columnas se separan con espacios y las filas con punto y coma. Por ejemplo, `[1 2; 3 4]` define una matriz de tamaño 2×2 . Mientras que los vectores se obtienen separando los elementos con comas , o punto y coma ;, como en `[1,2,3]` o `[1;2;3]`.

Por esta razón la expresión `[1 2 3]` produce una matriz de dimensión 1×3 .

1.3.4. Matriz cero (Zero matrix)

La matriz cero es aquella cuyos elementos son todos iguales a cero. Es el elemento neutro de la suma matricial.

Matriz cero en Julia

Podemos construir la matriz cero de dimensión $m \times n$ utilizando la función `zeros()`. La sintaxis general es:

```
zeros(T, (m, n))
```

donde:

- T corresponde al tipo de dato de los elementos de la matriz.
- m corresponde al número de filas.
- n corresponde al número de columnas.

Si omitimos el tipo T , es decir, si declaramos la función como `zeros(m,n)`, tendremos una matriz $m \times n$ llena de ceros cuyos elementos son ceros de tipo `Float64` por defecto.

```
Z = zeros(3,3)      # matriz 3x3 llena de ceros
```

```
-----  
3x3 Matrix{Float64}:
```

```
0.0  0.0  0.0  
0.0  0.0  0.0  
0.0  0.0  0.0
```

1.3.5. Matriz identidad (Identity matrix)

La matriz identidad es una matriz cuadrada cuyos elementos de la diagonal principal son iguales a 1 y los demás elementos son 0.

Matriz identidad en Julia

Podemos construir matrices identidad de varias formas, las más recomendadas son utilizando la función `Matrix()` de la siguiente manera:

```
Matrix{I,n,n}      # genera la matriz identidad de tamaño nxn
```

```
using LinearAlgebra
I3 = Matrix{I, 3, 3}    # matriz identidad 3x3
```

```
-----
3x3 Matrix{Bool}:
```

```
 1  0  0
 0  1  0
 0  0  1
```

Otra forma interesante sería creando el operador identidad de tamaño n mediante `I(n)` y luego convertirlo en una matriz utilizando la función `Matrix()`

```
using LinearAlgebra
I3 = I{3}      #Operador Identidad de tamaño n
Matrix{I3}     #Conversion a matriz
```

```
-----
3x3 Matrix{Bool}:
```

```
 1  0  0
 0  1  0
 0  0  1
```

Es necesario cargar la librería `LinearAlgebra`

1.4. Operaciones con Vectores y Matrices

Representación interna en Julia

En Julia, tanto los vectores como las matrices son casos particulares de la estructura general denominada `Array`.

Un arreglo en Julia se escribe como:

$$\text{Array}\{T, N\}$$

donde:

- `T` representa el tipo de dato almacenado.
- `N` indica el número de dimensiones del arreglo.

Ejemplo

- `Array{Int64,1}` corresponde a un arreglo unidimensional de enteros.
- `Array{Float64,2}` corresponde a un arreglo bidimensional de números reales en punto flotante.

En Julia existen abreviaciones:

$$\text{Vector}\{T\} \equiv \text{Array}\{T, 1\}$$
$$\text{Matrix}\{T\} \equiv \text{Array}\{T, 2\}$$

Por lo tanto:

$$\text{Vector}\{\text{Int64}\}$$

indica un arreglo unidimensional cuyos elementos son enteros de 64 bits, mientras que

$$\text{Matrix}\{\text{Float64}\}$$

representa un arreglo bidimensional de números reales en punto flotante.

1.4.1. Operaciones con Vectores

1.4.1.1. Suma de vectores

La suma de vectores se realiza componente a componente. Geométricamente, corresponde a la regla del paralelogramo.

```
v = [1, 2, 3]
```

```
w = [4, 5, 6]
```

```
v + w
```

```
-----  
3-element Vector{Int64}:
```

```
5
```

```
7
```

```
9
```

La salida

```
3-element Vector{Int64}
```

puede interpretarse como:

$\underbrace{3\text{-element}}$	$\underbrace{\text{Vector}}$	$\underbrace{\{Int64\}}$
dimensión	estructura	tipo de dato

Esto representa un vector de dimensión 3, es decir, un elemento de $\mathbb{R}^3$, donde cada componente es un entero almacenado con precisión de 64 bits.

1.4.1.2. Producto por escalar

El producto por escalar consiste en multiplicar cada componente del vector por un escalar $\lambda \in \mathbb{R}$.

```
v = [1, 2, 3]
3v #tambien podemos utilizar el operador *, es decir 3*v
```

```
-----
3-element Vector{Int64}:
 3
 6
 9
```

La salida

```
3-element Vector{Int64}
```

indica que el resultado sigue siendo un vector de dimensión 3, con la misma estructura unidimensional y el mismo tipo de dato.

Observación

El producto por escalar no altera la dimensión del vector, sino únicamente su magnitud.

Si el escalar fuera un número decimal, por ejemplo si tomamos $\lambda = \mathbf{3.0}$, Julia cambiaría automáticamente el tipo del resultado a `Float64`.

```
v = [1, 2, 3]
3.0*v
```

```
-----
3-element Vector{Float64}:
 3.0
 6.0
 9.0
```

1.4.1.3. Producto punto (Dot product)

El producto punto entre dos vectores produce un escalar.

Sean $v, w \in \mathbb{R}^n$ vectores: Entonces el **producto punto** $\cdot$ entre dos vectores v y w se define como

$$v \cdot w = v_1 w_1 + v_2 w_2 + \cdots + v_n w_n = \sum_{i=1}^n v_i w_i.$$

Equivalentemente:

$$v \cdot w = v^\top w = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}.$$

Con Julia podemos calcular el producto punto de dos vectores

Sean v y w vectores en $\mathbb{R}^n$. La sintaxis general es

```
dot(v, w)
```

```
v = [1, 2, 3]
w = [4, 5, 6]
dot(v, w)
```

32

Geométricamente, el producto punto entre dos vectores u y v puede expresarse como

$$u \cdot v = \|u\| \|v\| \cos(\theta),$$

donde $\|u\|$ y $\|v\|$ representan las normas de los vectores u y v , respectivamente, y θ es el ángulo entre ellos.

1.4.2. Operaciones con Matrices

1.4.2.1. Suma de matrices

La suma de matrices se realiza elemento a elemento. Para que la operación esté definida, ambas matrices deben tener las mismas dimensiones.

```
A = [1 2; 3 4]
B = [5 6; 7 8]
```

```
A + B
```

```
-----
2x2 Matrix{Int64}:
 6  8
10 12
```

1.4.2.2. Producto matriz-escalar

La multiplicación de una matriz por un escalar se define de forma análoga a la multiplicación escalar de vectores. Esta operación consiste en multiplicar cada una de las entradas de la matriz por el escalar.

Si $A \in \mathbb{R}^{m \times n}$ y $\lambda \in \mathbb{R}$, entonces

$$\lambda A \in \mathbb{R}^{m \times n}.$$

```
A = [1 2; 3 4]
3*A
```

```
-----
2x2 Matrix{Int64}:
 3  6
 9 12
```

1.4.2.3. Producto matricial

El producto de matrices no se realiza elemento a elemento. Se define como una combinación lineal entre las filas de la primera matriz y las columnas de la segunda.

Si $A \in \mathbb{R}^{m \times n}$ y $B \in \mathbb{R}^{n \times p}$, entonces

$$C = AB \in \mathbb{R}^{m \times p}.$$

donde cada entrada de C está dada por

$$C_{ij} = \sum_{k=1}^n a_{ik}b_{kj} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}.$$

```
A = [1 2; 3 4]
```

```
B = [5 6; 7 8]
```

```
A * B
```

```
-----  
2x2 Matrix{Int64}:  
19  22  
43  50
```

1.4.3. Operadores en Julia

Los operadores permiten realizar operaciones matemáticas, lógicas y de comparación entre valores. En Julia, muchos operadores están diseñados para facilitar el cálculo numérico y el álgebra lineal.

1.4.3.1. Operadores Aritméticos

Operador	Descripción
+	Suma
-	Resta
*	Multiplicación
/	División
\	Resolver sistema lineal ($Ax = b$)
%	Módulo
^	Potencia

Ejemplo:

```
a = 10
b = 3

a + b      # 13
a / b      # 3.333...
a % b      # 1
```

1.4.3.2. Operadores de Comparación

Operador	Significado
==	Igualdad
!=	Diferente
<	Menor que
>	Mayor que
<=	Menor o igual
>=	Mayor o igual

Ejemplo:

```
5 == 5      # true
3 != 4      # true
```

1.4.3.3. Operadores Lógicos

Operador	Significado
&&	AND lógico
	OR lógico
!	Negación

Ejemplo:

```
x = 5
(x > 0) && (x < 10)  # true
```

Capítulo 2

Librería LinearAlgebra

El módulo `LinearAlgebra` forma parte de la biblioteca estándar de Julia y proporciona herramientas fundamentales para el trabajo con vectores, matrices y operaciones de álgebra lineal.

Aunque Julia permite crear arreglos sin importar módulos adicionales, muchas operaciones especializadas tales como factorizaciones, normas, determinantes e inversas, se encuentran definidas dentro de `LinearAlgebra`.

Para utilizar estas funcionalidades es necesario cargar el módulo de la siguiente manera:

```
using LinearAlgebra
```

El módulo incluye, entre otras, las siguientes funcionalidades:

- Operaciones matriciales: `inv`, `det`, `rank`.
- Normas y productos internos: `norm`, `dot`.
- Factorizaciones: LU, QR, SVD.
- Tipos estructurados: `Diagonal`, `Symmetric`, `UpperTriangular`, `LowerTriangular`.

2.1. Sistema de ayuda en Julia

Julia incorpora un sistema de documentación interactivo que permite consultar la sintaxis y descripción de funciones, tipos y operadores directamente desde la consola.

La forma general para acceder a la ayuda es:

```
?nombre_de_función
```

Por ejemplo, si una función se llama `function`, la consulta se realiza escribiendo:

```
?function
```

Al ejecutar este comando, Julia muestra:

- La firma general de la función.
- Los parámetros que recibe.
- La descripción de su comportamiento.
- Ejemplos de uso (si se encuentran documentados).

En términos generales, la sintaxis de una función suele mostrarse como:

```
function(arg1, arg2, ..., argN)
```

donde los argumentos pueden ser obligatorios u opcionales, dependiendo de como se haya definido dicha función.

El sistema de ayuda también puede utilizarse para consultar:

- Tipos de datos: `?Matrix`
- Operadores: `?+`
- Constantes: `?pi`

Este mecanismo permite explorar la biblioteca estándar de manera segura, evitando errores de sintaxis y facilitando el aprendizaje del lenguaje.

2.2. Operadores para Álgebra Lineal

Julia distingue entre multiplicación matricial y operaciones elemento a elemento, además de incluir operadores y funciones específicas para vectores y matrices.

Operador / función	Descripción
<code>*</code>	Producto matricial
<code>.*</code>	Multiplicación elemento a elemento
<code>/</code>	División matricial por la derecha
<code>\</code>	Resolución de sistema lineal ($Ax = b$)
<code>./</code>	División elemento a elemento
<code>.\</code>	División elemento a elemento inversa
<code>^</code>	Potencia matricial (A^k)
<code>.^</code>	Potencia elemento a elemento
<code>dot(x,y)</code>	Producto punto
<code>cross(x,y)</code>	Producto cruz en $\mathbb{R}^3$
<code>norm(x)</code>	Norma de un vector

Ejemplo:

```
A = [1 2; 3 4]
B = [2 0; 1 2]

A * B      # Producto matricial
A .* B     # Multiplicación elemento a elemento

-----

2x2 Matrix{Int64}:
 4  4
10  8

2x2 Matrix{Int64}:
 2  0
 3  8
```

Otros operadores importantes:

```
A'          # Transpuesta conjugada
transpose(A) # Transpuesta
inv(A)       # Inversa
det(A)       # Determinante
eigvals(A)   # Autovalores
```

2.2.1. Norma de un vector

Sea $v \in \mathbb{R}^n$ un vector:

$$v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}.$$

La norma euclidea de v mide su longitud en el espacio y se define como

$$\|v\| = \sqrt{v \cdot v} = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2} = \sqrt{\sum_{i=1}^n v_i^2}.$$

Observación

La norma euclidiana también se escribe frecuentemente utilizando el subíndice 2. Es decir, si $v \in \mathbb{R}^n$, entonces

$$\|v\|_2 = \|v\|$$

El subíndice 2 indica que las componentes del vector se elevan al cuadrado antes de sumarse.

Con Julia podemos calcular la norma de un vector

Sea $v \in \mathbb{R}^n$ un vector. La sintaxis general es

```
norm(v)
```

Nota: Es necesario importar la librería `LinearAlgebra`.

```
using LinearAlgebra
```

```
v = [1, 2, 3]
```

```
norm(v)
```

```
-----  
3.7416573867739413
```

El resultado es un escalar de tipo `Float64`, ya que la operación involucra una raíz cuadrada.

2.2.2. Producto cruz

El producto cruz está definido únicamente en $\mathbb{R}^3$. Produce un vector perpendicular a los vectores originales.

Si $u, v \in \mathbb{R}^3$, entonces puede expresarse formalmente como

$$u \times v = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{vmatrix}.$$

equivalentemente:

$$u \times v = (u_2v_3 - u_3v_2)\mathbf{i} - (u_1v_3 - u_3v_1)\mathbf{j} + (u_1v_2 - u_2v_1)\mathbf{k}.$$

donde $\mathbf{i}, \mathbf{j}, \mathbf{k}$ son los vectores unitarios canónicos de $\mathbb{R}^3$, definidos como

$$\mathbf{i} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{j} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{k} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

```
using LinearAlgebra
```

```
u = [1, 2, 3]
```

```
v = [4, 5, 6]
```

```
cross(u, v)
```

```
-----  
3-element Vector{Int64}:
```

```
-3
```

```
6
```

```
-3
```

El resultado es un vector en $\mathbb{R}^3$ perpendicular a los vectores originales.

2.2.3. Norma de una matriz

La norma de una matriz mide su magnitud de forma análoga a la norma de un vector. Si $A \in \mathbb{R}^{m \times n}$, entonces la **norma de la matriz** se define como

$$\|A\| = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2}.$$

Nota:

En contextos donde se utilizan otras normas de matrices, esta norma se conoce como la **norma de Frobenius** y se denota como

$$\|A\|_F.$$

Además, se cumple que

$$\|A\|^2 = \|a_1\|^2 + \|a_2\|^2 + \cdots + \|a_n\|^2,$$

donde $a_1, \dots, a_n$ son las columnas de A .

```
using LinearAlgebra
```

```
A = [1 2; 3 4]  
norm(A)
```

```
-----  
5.477225575051661
```

2.2.4. Transpuesta de una matriz

La transpuesta de una matriz se obtiene intercambiando filas por columnas.

$$(A^T)_{ij} = A_{ji}.$$

```
using LinearAlgebra

A = [1 2; 3 4]
A'
```

```
2x2 adjoint(::Matrix{Int64}) with eltype Int64:
 1  3
 2  4
```

Nota:

En Julia, el operador `'` produce el *adjoint*. Para matrices reales coincide con la transpuesta. Para matrices complejas incluye conjugación.

2.2.5. Inversa de una matriz

Una matriz cuadrada A es invertible si existe una matriz A^{-1} tal que

$$AA^{-1} = I.$$

```
using LinearAlgebra

A = [1 2; 3 4]
inv(A)
```

```
2x2 Matrix{Float64}:
-2.0  1.0
 1.5 -0.5
```

Observación importante:

Aunque la matriz original estaba definida como

`Matrix{Int64}`,

la inversa aparece como

`Matrix{Float64}`.

Esto ocurre porque la inversa de una matriz generalmente produce números no enteros. Sea A una matriz 2×2 definida como:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}.$$

Si $\det(A) \neq 0$, su inversa está dada por

$$A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Dado que el determinante aparece en el denominador, los resultados suelen involucrar fracciones, por lo que Julia promueve automáticamente el tipo a `Float64`.

2.3. Consideraciones prácticas y ejemplos

En esta sección se presentan algunas consideraciones prácticas sobre la construcción y manipulación de vectores y matrices en Julia.

Además, se incluyen ejemplos que ilustran el comportamiento de distintas estructuras de datos y operaciones comunes en álgebra lineal.

Números complejos y forma (Vector vs Matrix)

En Julia, la forma en que se separan los elementos dentro de los corchetes determina si se construye un vector o una matriz.

Además, cuando se usan números complejos, el tipo del arreglo (`ComplexF64`, `Complex{Int}`, etc.) depende de los literales utilizados.

Concatenación horizontal (espacios)

Separar elementos con espacios genera concatenación horizontal, es decir, una matriz fila.

```
x_row = [1.0+0im 1 1]    # Matriz 1x3
```

```
-----  
1x3 Matrix{ComplexF64}:  
 1.0+0.0im  1.0+0.0im  1.0+0.0im
```

Vector unidimensional (comas)

Separar elementos con comas construye un vector unidimensional.

```
x_vec = [1.0+0im, 1, 1]    # Vector de 3 elementos
```

```
-----  
3-element Vector{ComplexF64}:  
 1.0+0.0im  
 1.0+0.0im  
 1.0+0.0im
```

Concatenación vertical (;)

Separar con punto y coma genera concatenación vertical, es decir, una matriz columna.

```
x_col = [1.0+0im; 1; 1]    # Matriz 3x1
```

Observaciones importantes

- Espacios → concatenación horizontal (misma fila).
- ; → concatenación vertical (nuevas filas).
- , → vector unidimensional.
- Un **vector** no es lo mismo que una **matriz columna** en términos de tipo.

`/` vs `//` en Julia

La operación `/` produce números en punto flotante (`Float64`), mientras que `//` produce números racionales exactos (`Rational`).

```
a = 1/3  
b = 1//3
```

```
typeof(a)  
typeof(b)
```

```
-----  
(0.3333333333333333, Float64)  
(1//3, Rational{Int64})
```

En este caso, note que el tipo de `b` es `Rational{Int64}`, lo cual indica que el valor se almacena como un número racional exacto de la forma

$$\frac{p}{q},$$

donde p y q son números enteros representados con el tipo `Int64`.

Construcción de una matriz por comprensión

En Julia, una matriz puede construirse mediante comprensión utilizando la siguiente forma general:

$$A = [f(i, j) \text{ for } i \text{ in } I, j \text{ in } J] \quad \text{o equivalentemente} \quad A = [f(i, j) \text{ for } i = I, j = J].$$

donde: $f(i, j)$ es la función que define cada elemento, i recorre el conjunto de índices de filas I y j recorre el conjunto de índices de columnas J .

Ejemplo:

```
X = [i + j for i = 1:2, j = 3:4]
```

```
-----  
2x2 Matrix{Int64}:  
 4  5  
 5  6
```

La expresión

$$[i + j \text{ for } i = 1 : 2, j = 3 : 4]$$

significa que cada elemento se define como

$$X_{ij} = i + j,$$

con

$$i \in \{1, 2\}, \quad j \in \{3, 4\}.$$

Por lo tanto,

$$X = \begin{bmatrix} 1+3 & 1+4 \\ 2+3 & 2+4 \end{bmatrix} = \begin{bmatrix} 4 & 5 \\ 5 & 6 \end{bmatrix}.$$

Matriz definida por una función racional

Consideremos la matriz definida por

$$A_{ij} = \frac{1}{i+j}, \quad i, j = 1, \dots, 5.$$

En Julia puede construirse mediante comprensión.

```
A = [1 / (i + j) for i in 1:5, j in 1:5]
```

```
-----  
5x5 Matrix{Float64}:  
 0.5      0.333333  0.25      0.2       0.166667  
 0.333333  0.25     0.2       0.166667  0.142857  
 0.25     0.2      0.166667  0.142857  0.125  
 0.2      0.166667  0.142857  0.125     0.111111  
 0.166667  0.142857  0.125     0.111111  0.1
```

Inicialización de una matriz con ceros

En muchos algoritmos es necesario inicializar primero una matriz y luego modificar sus entradas mediante ciclos.

En Julia, la función `zeros(m,n)` crea una matriz de dimensión $m \times n$ cuyos elementos son todos cero.

```
A = zeros(5,5)
```

```
-----  
5x5 Matrix{Float64}:  
 0.0  0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0  0.0  
 0.0  0.0  0.0  0.0  0.0
```

En la salida

```
5x5 Matrix{Float64}
```

El tipo `Float64` aparece porque la función `zeros` crea por defecto números reales en punto flotante.

Construcción imperativa mediante ciclos

Además de la comprensión, una matriz puede construirse de forma imperativa utilizando ciclos anidados.

La estructura general para construir $A \in \mathbb{R}^{m \times n}$, donde cada entrada está definida por una función $f(i, j)$ con m filas y n columnas es

```
A = zeros(m,n)
for i in 1:m
    for j in 1:n
        A[i,j] = f(i,j)
    end
end
```

Ejemplo: Construyamos la matriz definida por:

$$A_{ij} = \frac{1}{i+j}, \quad 1 \leq i, j \leq 5.$$

```
A = zeros(5,5)
for i in 1:5
    for j in 1:5
        A[i,j] = 1 / (i + j)
    end
end

-----
5x5 Matrix{Float64}:
 0.5      0.333333  0.25      0.2       0.166667
 0.333333  0.25     0.2       0.166667  0.142857
 0.25     0.2      0.166667  0.142857  0.125
 0.2      0.166667  0.142857  0.125     0.111111
 0.166667  0.142857  0.125     0.111111  0.1
```

Este método es equivalente a la construcción por comprensión:

```
A = [1/(i+j) for i=1:5, j=1:5]
```

Observación:

El enfoque imperativo:

- Es más explícito y tradicional.
- Permite mayor control del flujo (condiciones, actualizaciones parciales, etc.).
- Se asemeja más a la implementación clásica en lenguajes como C o Fortran.

Extracción de submatrices (indexación por rangos)

En Julia, la selección de submatrices se realiza mediante la notación general

$$A[\text{filas}, \text{columnas}],$$

Ejemplo

Consideremos la matriz definida por

$$A_{ij} = \frac{1}{i+j}, \quad i, j = 1, \dots, 5.$$

```
A = [1 / (i + j) for i = 1:5, j = 1:5]
```

```
-----  
5x5 Matrix{Float64}:
```

```
0.5      0.333333  0.25      0.2       0.166667  
0.333333  0.25     0.2       0.166667  0.142857  
0.25     0.2      0.166667  0.142857  0.125  
0.2      0.166667  0.142857  0.125     0.111111  
0.166667  0.142857  0.125     0.111111  0.1
```

Caso 1: Todas las filas, algunas columnas

Para seleccionar todas las filas y únicamente las primeras tres columnas:

```
B = A[:, 1:3]
```

```
-----  
5x3 Matrix{Float64}:  
 0.5      0.333333  0.25  
 0.333333  0.25     0.2  
 0.25     0.2      0.166667  
 0.2      0.166667  0.142857  
 0.166667  0.142857  0.125
```

Caso 2: Subconjunto de filas y columnas

También es posible seleccionar subconjuntos específicos:

```
C = A[2:4, 2:5]
```

```
-----  
3x4 Matrix{Float64}:  
 0.25     0.2      0.166667  0.142857  
 0.2      0.166667  0.142857  0.125  
 0.166667  0.142857  0.125     0.111111
```

Capítulo 3

Error Numérico y Problemas Mal Condicionados

Se seguirá principalmente el tratamiento desarrollado por Trefethen y Bau en *Numerical Linear Algebra* [3], lecciones 12–19, donde se introducen los conceptos de aritmética de punto flotante, error de redondeo, número de condición y estabilidad numérica.

3.1. Error de representación en punto flotante

Muchas veces asumimos que los cálculos realizados en un computador son tan sencillos como hacer una operación a mano. Por ejemplo, si sumamos

$$0.1 + 0.2$$

esperamos obtener exactamente

$$0.3$$

Ahora, ¿qué sucede si realizamos esta misma operación en el computador? Probemos en Julia:

```
0.1 + 0.2
-----
0.30000000000000004
```

En efecto, obtenemos

$$0.30000000000000004$$

pero, ¿por qué ocurre esto?

La realidad es que el problema es más complejo de lo que parece.

Como señalan Trefethen y Bau [3]:

“En un computador, los números reales se representan utilizando un número finito de bits, por lo que sólo un subconjunto finito de los números reales puede almacenarse exactamente. Esta limitación presenta dos dificultades. En primer lugar, los números representados no pueden ser arbitrariamente grandes o pequeños. En segundo lugar, deben existir espacios entre ellos.”

Consideremos la siguiente expansión decimal

0.333333333333...

la cual representa exactamente el número $1/3$, aunque su expansión decimal sea infinita, $1/3$ en binario se vería así

$$\frac{1}{3} = 0.0101010101..._2$$

es decir, su representación binaria también es infinita y periódica.

Como el computador utiliza un número fijo de bits para representar los números, muchos de estos deben ser aproximados cuando se almacenan en memoria.

Por esta razón, vemos que **0.1**, **0.2** y muchos más números no se pueden representar de manera exacta en su representación binaria.

Este problema se conoce como **error de representación en punto flotante** (*floating-point representation error*) y, como vimos anteriormente es una consecuencia directa de la precisión finita con la que los computadores almacenan los números.

3.2. Error de redondeo

Dado que los números se representan con precisión finita en un computador, muchos números reales deben ser aproximados cuando se almacenan en memoria. Esta aproximación introduce una pequeña diferencia entre el valor real de un número y el valor que realmente se almacena en el sistema de punto flotante. A esta diferencia se le conoce como **error de redondeo**.

Para cuantificar este error se introduce una constante conocida como **epsilon de máquina** (*machine epsilon*), la cual mide la precisión del sistema de punto flotante.

De manera intuitiva, $\epsilon_{machine}$ representa la distancia relativa entre el número 1 y el siguiente número representable en el sistema de punto flotante.

Además del redondeo al almacenar números, también se introducen errores cuando se realizan operaciones aritméticas (+, −, × y ÷).

En aritmética de punto flotante se cumple una propiedad fundamental, conocida como el **axioma fundamental de la aritmética de punto flotante** [3], el cual establece que:

Para todo $x, y \in F$, existe un número ε tal que $|\varepsilon| \leq \epsilon_{machine}$ y

$$x \circledast y = (x * y)(1 + \varepsilon).$$

donde F es el conjunto de números de punto flotante (*floating point numbers*), es decir, el conjunto de números que pueden representarse exactamente en el sistema de punto flotante del computador, $*$ representa la operación exacta y $\circledast$ su equivalente en aritmética de punto flotante.

“Esto significa que cada operación aritmética realizada por el computador es exacta salvo por un pequeño error relativo.”

3.3. Propagación del error

Como vimos anteriormente, los números en un computador se representan, con precisión finita. Esto implica que cada vez que un número se almacena o se realiza una operación aritmética, se induce un pequeño error de redondeo.

Cuando un cálculo involucra múltiples operaciones, estos pequeños errores no permanecen aislados. Por el contrario, los resultados obtenidos en una operación suelen utilizarse en operaciones posteriores.

Como consecuencia, los errores inducidos en una etapa del cálculo pueden propagarse a lo largo de todo el proceso computacional. Este fenómeno se conoce como **propagación del error**. En general, los errores pueden acumularse o incluso amplificarse dependiendo de la estructura del problema y del algoritmo utilizado.

En muchos casos, los errores siguen siendo pequeños y no afectan de manera significativa el resultado final. Sin embargo, en ciertos problemas numéricos los errores pueden crecer considerablemente a tal punto de producir resultados altamente imprecisos.

Por eso un pequeño error inicial puede propagarse y afectar el resultado final del cálculo.

3.4. Error de cancelación numérica

Un fenómeno importante que puede aparecer en los cálculos numéricos es la **cancelación numérica**. Este ocurre cuando se restan dos números muy cercanos entre sí.

Cuando dos números tienen valores muy similares, muchos de sus dígitos significativos se cancelan durante la resta. Como consecuencia, el resultado final depende principalmente de los últimos dígitos disponibles.

Por ejemplo, consideremos el siguiente cálculo en Julia:

```
x = 1.0000001
y = 1.0000000
x - y
-----
1.0000000005838672e-7
```

En este caso, los primeros dígitos de ambos números se cancelan durante la resta. Como resultado, el valor final depende únicamente de los últimos dígitos disponibles, los cuales pueden estar afectados por errores de redondeo. Si estos últimos dígitos contienen errores de redondeo, dichos errores pueden afectar significativamente el resultado del cálculo, produciendo una pérdida considerable de precisión.

3.5. Problemas mal condicionados

En muchos casos la magnitud del error final no depende únicamente del algoritmo utilizado, sino también de la naturaleza del problema que se está resolviendo.

En análisis numérico, esta sensibilidad se estudia mediante el concepto de **condicionamiento**. El condicionamiento de un problema mide cómo pequeñas perturbaciones en los datos de entrada pueden afectar la solución del problema.

Un problema se dice **bien condicionado** si pequeñas variaciones en los datos producen únicamente pequeñas variaciones en la solución. Por el contrario, un problema se dice **mal condicionado** si pequeñas perturbaciones en los datos pueden producir cambios significativos en el resultado.

Veamos 2 ejemplos en Julia.

```
f(x) = 2x
x = 1.0
y = 1e-6
f(x)
f(x + y)
```

```
-----
2.0
2.000002
```

En este caso, una pequeña perturbación en la entrada produce un cambio también pequeño en el resultado. Este es un ejemplo de un problema bien condicionado.

En análisis numérico, una **perturbación** es una pequeña variación o modificación en los datos de entrada de un problema, como por ejemplo un pequeño cambio en los valores de una matriz o de un vector.

Por ejemplo, si en lugar de trabajar con un valor exacto b utilizamos un valor ligeramente diferente $b + \delta b$, donde δb es una cantidad pequeña, estamos introduciendo una perturbación en los datos del problema.

```
g(x) = 1/x
```

```
x = 1e-6
```

```
y = 1e-7
```

```
g(x)
```

```
g(x + y)
```

```
g(x + y) - g(x)
```

```
-----  
1.0e6
```

```
909090.9090909091
```

Aquí observamos que una perturbación pequeña en el valor de entrada produce un cambio mucho mayor en el resultado. Este comportamiento es característico de un problema mal condicionado.

En álgebra lineal numérica, algunos de los problemas cuyo condicionamiento es de especial interés incluyen la resolución de sistemas lineales, el cálculo de autovalores y autovectores, y los problemas de mínimos cuadrados.

Ejemplo: solución numérica de un sistema lineal

Consideremos el sistema lineal

$$Ax = b.$$

En muchos casos, incluso cuando el sistema tiene una solución exacta conocida, la solución obtenida en un computador no coincide exactamente con ella debido a los errores numéricos introducidos por la aritmética de punto flotante.

Para ilustrar este problema, construiremos un sistema lineal cuya solución exacta sea conocida de antemano. Luego resolveremos el sistema en Julia y compararemos la solución numérica obtenida con la solución verdadera.

Tomemos la matriz

$$A = \begin{bmatrix} \frac{1}{1+1} & \frac{1}{1+2} & \frac{1}{1+3} & \frac{1}{1+4} & \frac{1}{1+5} \\ \frac{1}{2+1} & \frac{1}{2+2} & \frac{1}{2+3} & \frac{1}{2+4} & \frac{1}{2+5} \\ \frac{1}{3+1} & \frac{1}{3+2} & \frac{1}{3+3} & \frac{1}{3+4} & \frac{1}{3+5} \\ \frac{1}{4+1} & \frac{1}{4+2} & \frac{1}{4+3} & \frac{1}{4+4} & \frac{1}{4+5} \\ \frac{1}{5+1} & \frac{1}{5+2} & \frac{1}{5+3} & \frac{1}{5+4} & \frac{1}{5+5} \end{bmatrix}$$

y la solución exacta

$$x_{\text{verdadero}} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

A partir de estos datos, construimos

$$b = Ax_{\text{verdadero}}.$$

```
using LinearAlgebra

A = [1/(i+j) for i in 1:5, j in 1:5]
x_verdadero = ones(5)
b = A * x_verdadero
x_calculado = A \ b

-----
5-element Vector{Float64}:
 0.9999999999987796
 1.0000000000142275
 0.9999999999513124
 1.0000000000637648
 0.9999999999717184
```

Al resolver el sistema en Julia obtenemos una solución aproximada $x_{\text{calculado}}$.

Idealmente, esta solución debería coincidir exactamente con $x_{\text{verdadero}}$. Sin embargo, debido a la aritmética de punto flotante, aparecen pequeñas diferencias entre ambas.

```
error_absoluto = norm(x_calculado - x_verdadero)
error_relativo = norm(x_calculado - x_verdadero) / norm(x_verdadero)
-----
8.62565330017277e-11
3.857509425906342e-11
```

Para cuantificar la diferencia entre la solución exacta y la solución numérica, calculamos el error absoluto

$$\|x_{\text{calculado}} - x_{\text{verdadero}}\|$$

y el error relativo

$$\frac{\|x_{\text{calculado}} - x_{\text{verdadero}}\|}{\|x_{\text{verdadero}}\|}.$$

Estas cantidades permiten medir qué tan cerca se encuentra la solución numérica de la solución exacta.

De esta manera, podemos observar que, aunque el sistema haya sido construido a partir de una solución exacta conocida, la solución obtenida numéricamente no coincide de forma exacta con ella.

Esto muestra cómo los errores numéricos pueden aparecer incluso en problemas aparentemente simples.

Ejemplo: solución mediante mínimos cuadrados

Consideremos ahora un sistema lineal donde el número de ecuaciones es mayor que el número de incógnitas.

En este caso el sistema generalmente no tiene una solución exacta.

Por ejemplo, construyamos una matriz $A \in \mathbb{R}^{10 \times 3}$ y un vector $b \in \mathbb{R}^{10}$.

```
using LinearAlgebra
A = rand(10,3)
b = rand(10)
x = A \ b

-----
3-element Vector{Float64}:
 0.3989985034326874
-0.16006185426988806
 0.5205984063533916
```

La función `rand(m,n)` genera una matriz de tamaño $m \times n$ cuyos elementos son números aleatorios distribuidos uniformemente en el intervalo $[0, 1)$.

En Julia, la operación

$$x = A \backslash b$$

resuelve el sistema lineal $Ax = b$. Cuando la matriz A es cuadrada, Julia calcula la solución exacta del sistema. Sin embargo, cuando el sistema es **sobredeterminado** (es decir, cuando el número de ecuaciones es mayor que el número de incógnitas), como en este ejemplo, Julia calcula automáticamente la **solución de mínimos cuadrados**.

Es decir, se obtiene el vector x que minimiza el error

$$\|Ax - b\|.$$

En Julia podemos medir el error residual de la siguiente manera

```
norm(A*x - b)

-----
0.5526704437550841
```

El valor obtenido corresponde a la norma del vector residual

$$r = Ax - b,$$

que mide qué tan cerca está la solución calculada de satisfacer el sistema original.

En un sistema sobredeterminado este residuo generalmente no es cero, ya que no existe una solución exacta que satisfaga todas las ecuaciones simultáneamente.

Si el residuo es cercano a cero, significa que el vector x aproxima bien una solución del sistema en el sentido de mínimos cuadrados.

Si el residuo es exactamente cero, entonces el sistema tiene una solución exacta y el vector x satisface todas las ecuaciones simultáneamente.

Por el contrario, si el residuo es relativamente grande, esto indica que el sistema no puede satisfacerse exactamente y que la solución obtenida sólo representa un compromiso entre las diferentes ecuaciones del sistema.

Capítulo 4

Transformaciones Lineales y Afines

En muchas aplicaciones geométricas y computacionales es necesario describir transformaciones que modifican la posición o la forma de objetos en el plano o en el espacio. Estas transformaciones pueden representarse mediante funciones que actúan sobre vectores en $\mathbb{R}^n$.

La presentación de las transformaciones afines que se desarrolla en esta sección sigue principalmente la exposición de Olver y Shakiban en [6]. En dicho texto se estudian las transformaciones afines como aplicaciones que incluyen diversas transformaciones geométricas como rotaciones, reflexiones, traslaciones, escalamiento y cizallamiento. Las matrices asociadas a rotaciones y reflexiones que se utilizan siguen la formulación presentada en *Introduction to Applied Linear Algebra* de Boyd y Vandenberghe [5].

Una función

$$T : \mathbb{R}^n \rightarrow \mathbb{R}^n$$

de la forma

$$T(x) = Ax + b$$

donde A es una matriz $n \times n$ y $b \in \mathbb{R}^n$, se denomina **transformación afín**.

En esta expresión, la matriz A representa la parte lineal de la transformación, mientras que el vector b representa una traslación

Las transformaciones afines preservan varias propiedades geométricas importantes, entre ellas el paralelismo entre rectas, las proporciones entre segmentos y la estructura geométrica general de las figuras.

Sin embargo, a diferencia de las transformaciones lineales, una transformación afín no necesariamente preserva el origen.

En las siguientes secciones se estudiará cómo representar estas transformaciones en el plano y en el espacio, así como su implementación computacional utilizando Julia.

4.1. Transformaciones afines en 2D y 3D

En el plano, una transformación afín tiene la forma

$$F(x) = Ax + b$$

con

$$A \in \mathbb{R}^{2 \times 2}, \quad b \in \mathbb{R}^2$$

mientras que en el espacio tridimensional

$$A \in \mathbb{R}^{3 \times 3}, \quad b \in \mathbb{R}^3.$$

Esto permite transformar puntos del plano o del espacio mediante operaciones matriciales.

Ilustración de una transformación afín

Consideremos el punto

$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}.$$

Sea además

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Primero se calcula la parte lineal

$$Ax = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 8 \\ 3 \end{bmatrix}.$$

Luego se suma el vector de traslación

$$T(x) = Ax + b = \begin{bmatrix} 8 \\ 3 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 9 \\ 2 \end{bmatrix}.$$

Por lo tanto, el punto $(2, 3)$ se transforma en $(9, 2)$.

En Julia tenemos lo siguiente:

```
using Plots

A = [1.0 2.0;
     0.0 1.0]

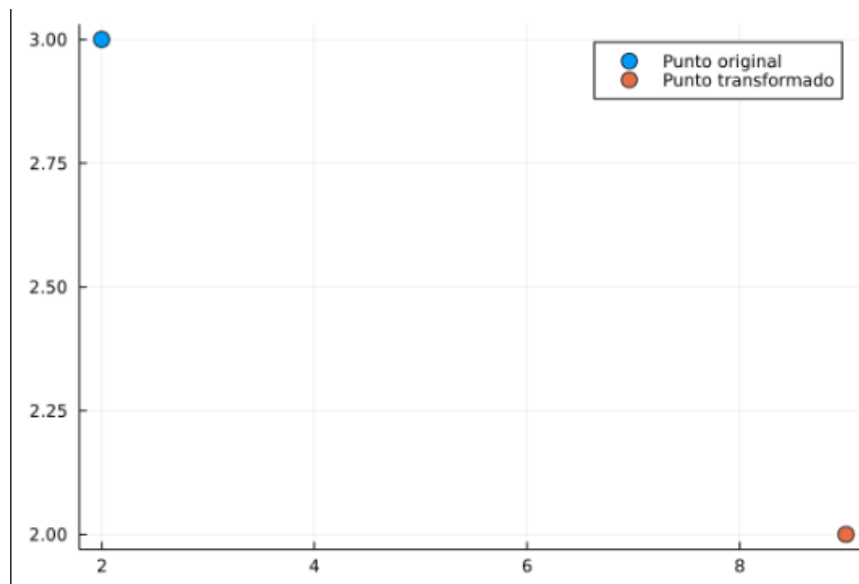
b = [1.0, -1.0]

x = [2.0, 3.0]

y = A*x + b

x, y
-----
([2.0, 3.0], [9.0, 2.0])
```

```
scatter([x[1]], [x[2]], label="Punto original", markersize=6)
scatter!([y[1]], [y[2]], label="Punto transformado", markersize=6)
```



En Julia, una transformación afín se aplica multiplicando el punto por la matriz A y luego sumando el vector b .

4.2. Isometrías

Una isometría es una transformación geométrica que preserva distancias entre puntos. Entre las isometrías más importantes se encuentran

- traslaciones
- rotaciones
- reflexiones

Estas transformaciones mantienen la forma y tamaño de las figuras.

4.2.1. Traslaciones

Una deslaza todos los puntos de una figura por un mismo vector. Si v es el vector de traslación, entonces

$$T(x) = x + v$$

4.2.2. Rotaciones

Una rotación en el plano alrededor del origen por un ángulo θ se describe mediante la matriz

$$R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

4.2.3. Reflexiones

Una reflexión invierte una figura respecto a una recta en el plano.

Si la recta de reflexión pasa por el origen y forma un ángulo θ con el eje horizontal, entonces la transformación puede representarse mediante la matriz

$$Ref_\theta = \begin{bmatrix} \cos(2\theta) & \sin(2\theta) \\ \sin(2\theta) & -\cos(2\theta) \end{bmatrix}.$$

4.3. Imágenes y figuras en Julia

En computación, una imagen rasterizada puede interpretarse como una matriz cuyos elementos representan píxeles. En una imagen en escala de grises, cada entrada de la matriz almacena un valor numérico asociado a la intensidad del píxel. En cambio, los gráficos vectoriales no se describen píxel por píxel, sino mediante objetos geométricos definidos por coordenadas y relaciones matemáticas, como puntos, segmentos y polígonos.

Haremos uso de la librería **plots** para poder dibujar graficos

```
using Plots
```

Ejemplo: imagen rasterizada como matriz

Una imagen **rasterizada** es una imagen representada por una rejilla rectangular de píxeles. En este caso, cada entrada de la matriz corresponde a un píxel de la imagen. En Julia, muchas imágenes pueden manipularse directamente como matrices. El siguiente ejemplo carga una imagen de prueba en escala de grises y extrae un recorte rectangular.

```
import Pkg; Pkg.add("TestImages")

using TestImages
using Plots

# cargar imagen de prueba en escala de grises
img = testimage("cameraman")
```



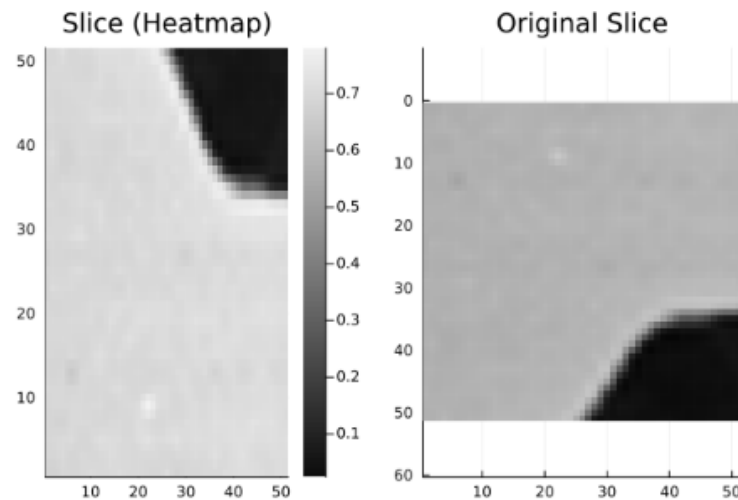
```

# tomar un recorte de la imagen (slice)
# las imágenes en Julia se comportan como matrices

recorte = Real.(img[100:150, 100:150]) # muestra la matriz

# Visualizar el recorte como heatmap para ver los valores numéricos
plot(
    heatmap(recorte, color=:grays, title="Slice (Heatmap)"),
    plot(img[100:150, 100:150], title="Original Slice"),
    layout=(1,2)
)

```



El recorte anterior puede interpretarse como una matriz cuyos elementos representan intensidades de gris.

4.4. Gráficos Vectoriales

A diferencia de las imágenes rasterizadas, los gráficos vectoriales no almacenan información píxel por píxel. En su lugar, describen las figuras mediante instrucciones matemáticas.

Entre los elementos básicos se encuentran:

- **Puntos:** definidos por coordenadas (x, y) .
- **Segmentos:** definidos por dos puntos.
- **Polígonos:** definidos por una lista de puntos conectados.

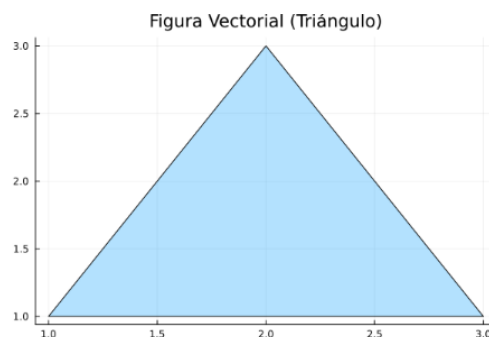
La **principal ventaja de los gráficos vectoriales** es que **pueden escalarse sin perder calidad**, ya que las figuras se describen mediante relaciones geométricas y no mediante píxeles individuales.

Ejemplo de gráfico vectorial

Podemos representar un polígono simple especificando las coordenadas de sus vértices.

```
using Plots
x = [1, 2, 3, 1] # Coordenadas en x
y = [1, 3, 1, 1] # Coordenadas en y

plot(x, y,
     seriestype=:shape,
     fillalpha=0.3,
     legend=false,
     title="Figura vectorial (triángulo)",
     markershape=:circle)
```



Estructuras de datos para figuras

Para trabajar de manera organizada con figuras vectoriales, podemos definir una jerarquía de tipos en Julia.

```
abstract type Figura end

struct Punto{T<:Real} <: Figura
    x::T
    y::T
end

struct Segmento <: Figura
    p1::Punto
    p2::Punto
end

struct Poligono <: Figura
    vertices::Vector{Punto}
end
```

En esta jerarquía:

- **Figura** es un tipo abstracto que representa cualquier figura.
- **Punto** representa una coordenada en el plano.
- **Segmento** está definido por dos puntos.
- **Poligono** representa una colección de vértices que forman una figura cerrada.

Ejercicio 1:

Crear una figura llamada **Circulo**

```
struct Circulo{T<:Real} <: Figura
    centro::Punto{T}
    radio::T
end
```

Creación de instancias

Podemos crear objetos de estos tipos de la siguiente manera:

```
p1 = Punto(1.0, 2.0)
p2 = Punto(3.0, 4.0)

seg = Segmento(p1, p2)
triangulo = Poligono([Punto(0,0), Punto(1,0), Punto(0,1)])
circ = Circulo(Punto(1.0,1.0), 2.0)
```

Visualización mediante *multiple dispatch*

Julia permite definir distintas implementaciones de una función según el tipo de los argumentos que recibe. Este mecanismo se conoce como *multiple dispatch*.

Podemos definir una función `dibujar` que se comporte de manera distinta dependiendo del tipo de figura.

```
function dibujar(p, fig::Punto; kwargs...)
    scatter!(p, [fig.x], [fig.y]; markershape=:circle, kwargs...)
end

function dibujar(p, fig::Segmento; kwargs...)
    plot!(p, [fig.p1.x, fig.p2.x], [fig.p1.y, fig.p2.y]; kwargs...)
end

function dibujar(p, fig::Poligono; kwargs...)
    xs = [v.x for v in fig.vertices]
    ys = [v.y for v in fig.vertices]

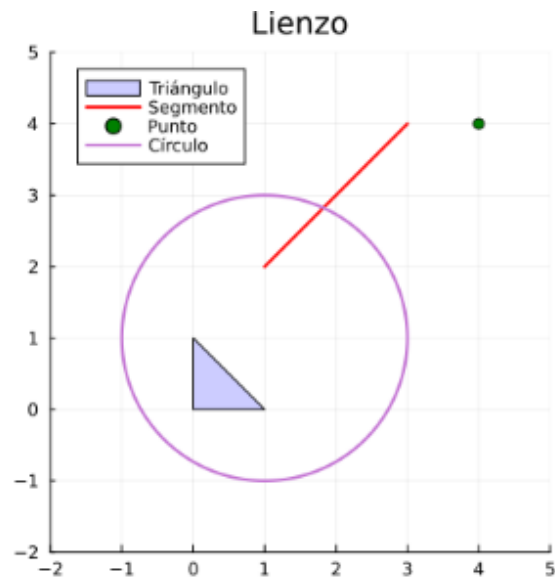
    push!(xs, fig.vertices[1].x)
    push!(ys, fig.vertices[1].y)
    plot!(p, xs, ys; seriestype=:shape, fillalpha=0.2, kwargs...)
end

function dibujar(p, fig::Circulo; N=200, kwargs...)
    h, k = fig.centro.x, fig.centro.y
    r = fig.radio
    theta = range(0, 2*pi, length=N)
    xs = h .+ r .* cos.(theta)
    ys = k .+ r .* sin.(theta)
    plot!(p, xs, ys; kwargs...)
end
```

Finalmente, podemos crear un lienzo y dibujar varias figuras en él.

```
mi_lienzo = plot(title="Lienzo",
                  aspect_ratio=:equal,
                  xlims=(-2,5),
                  ylims=(-2,5))

dibujar(mi_lienzo, triangulo, label="Triángulo", color=:blue)
dibujar(mi_lienzo, seg, label="Segmento", color=:red, lw=2)
dibujar(mi_lienzo, Punto(4,4), label="Punto", color=:green)
dibujar(mi_lienzo, circ, label="Círculo", lw=2)
mi_lienzo
```



La función `dibujar` ilustra el uso de *multiple dispatch* en Julia: se trata de una misma función conceptual, pero con distintas implementaciones dependiendo del tipo de figura recibido como argumento.

Coordenadas homogéneas

Para aplicar transformaciones afines en el plano, resulta conveniente usar coordenadas homogéneas. En este caso, un punto (x, y) se representa mediante el vector columna

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

En Julia:

```
# Definir la función genérica para coordenadas homogéneas
# Retorna una matriz de 3xN donde cada columna es [x, y, 1]

function coordenadas_homogeneas(p::Punto)
    return [p.x; p.y; 1.0][:, :]
end

function coordenadas_homogeneas(s::Segmento)
    return [s.p1.x s.p2.x;
            s.p1.y s.p2.y;
            1.0    1.0]
end

function coordenadas_homogeneas(poly::Poligono)

    N = length(poly.vertices)
    mat = ones(3, N)
    for i in 1:N
        mat[1, i] = poly.vertices[i].x
        mat[2, i] = poly.vertices[i].y
    end
    return mat
end
```

Ejercicio 2:

¿Es posible escribir la función `coordenadas_homogeneas` para un polígono (segmento) reutilizando la implementación previamente definida para puntos?

Notemos inicialmente que la función que tenemos definida de `coordenadas_homogeneas` para un segmento toma dos puntos, luego los embebe en $\mathbb{R}^3$ y nos devuelve una matriz conformada por dos columnas donde la primera columna, digamos `c1` representa el primer punto extremo y la segunda columna, digamos `c2` representa el otro punto extremo.

Ahora bien, como queremos reutilizar la implementación definida para puntos, podemos simplemente crear dos variables `c1` y `c2` en las cuales reutilizaremos la función `coordenadas_homogeneas` y pasaremos como parámetro cada punto del segmento. Recordemos que el segmento está compuesto por dos puntos `p1` y `p2`, a los cuales podemos acceder mediante `s.p1` y `s.p2`. Finalmente podemos realizar una concatenación horizontal de las columnas `c1` y `c2` utilizando la función `hcat()`, así tendríamos una nueva función utilizando la implementación definida para puntos:

```
function coordenadas_homogeneas(s::Segmento)

    c1 = coordenadas_homogeneas(s.p1)
    c2 = coordenadas_homogeneas(s.p2)
    return hcat(c1,c2)

end
```

De manera análoga, para el polígono, note que inicialmente se calcula el número N de vértices que tiene. Luego se crea una matriz de tamaño $3 \times N$ inicializada con unos e iteramos sobre esta matriz de la siguiente manera:

Recorremos cada vértice (punto) del polígono y en cada iteración se reemplazan los valores de las dos primeras filas de la columna i de la matriz por las coordenadas x e y del vértice correspondiente.

Ahora, como queremos reutilizar la implementación definida para puntos, podemos realizar casi que el mismo procedimiento solo que inicializaremos la matriz con ceros (podemos inicializarla con unos, no afectará en nada) y modificaremos el ciclo.

En este caso definimos una nueva variable digamos c la cual llamará a la función `coordenadas_homogeneas` pasando como parámetro el vértice i del conjunto de vértices del polígono, mediante `poly.vertices[i]`, esta nos devolverá el vértice embebido en $\mathbb{R}^3$ como un vector columna.

Finalmente, se reemplazan todas las filas de la columna i de la matriz por dicho vector, obteniendo así la matriz $3 \times N$ que contiene las coordenadas homogéneas de todos los vértices del polígono.

```
function coordenadas_homogeneas(poly::Poligono)
    N = length(poly.vertices)
    mat = zeros(3, N)

    for i in 1:N
        c = coordenadas_homogeneas(poly.vertices[i])
        mat[:,i] = c[:,1]
    end
    return mat
end
```

Reconstrucción de Figuras desde Coordenadas

Definamos una función que reconstruye una Figura a partir de su matriz de coordenadas homogéneas.

```
"""Función genérica para crear figuras desde la matriz
de coordenadas [x; y; 1]"""

function crear_figura(::Type{Punto}, m::Matrix)
    return Punto(m[1, 1], m[2, 1])
end

function crear_figura(::Type{Segmento}, m::Matrix)
    return Segmento(Punto(m[1, 1], m[2, 1]), Punto(m[1, 2], m[2, 2]))
end

function crear_figura(::Type{Poligono}, m::Matrix)
    n = size(m, 2)
    vertices = [Punto(m[1, i], m[2, i]) for i in 1:n]
    return Poligono(vertices)
end

dibujar(mi_lienzo, crear_figura(Poligono, [1 2 3; 0 2 1; 1 1 1]))
```



Definir matrices de transformación

En coordenadas homogéneas, las transformaciones afines en el plano pueden representarse mediante matrices de tamaño 3×3 . A continuación definimos funciones en Julia para generar matrices de traslación, rotación y escalado.

```
# Función para matriz de traslación
function matriz_traslacion(dx::Real, dy::Real)
    return Float64[1 0 dx; 0 1 dy; 0 0 1]
end
# Función para matriz de rotación (alrededor del origen)
function matriz_rotacion(theta::Real)
    c, s = cos(theta), sin(theta)
    return Float64[c -s 0;
                   s  c 0;
                   0  0 1]
end
# Función para matriz de escalado
function matriz_escalado(sx::Real, sy::Real)
    return Float64[sx 0 0;
                   0  sy 0;
                   0  0 1]
end
```

Aplicar transformaciones a figuras

Una vez definidas las matrices de transformación, podemos aplicarlas a las figuras utilizando sus coordenadas homogéneas. Para ello, definimos una función genérica **transformar**, la cual recibe una figura y una matriz de transformación, calcula las nuevas coordenadas y reconstruye la figura resultante.

```
function transformar(fig::T, M::Matrix) where T <: Figura
    coords_h = coordenadas_homogeneas(fig)
    nuevas_coords = M * coords_h
    return crear_figura(T, nuevas_coords)
end
```

En este caso, la transformación se aplica multiplicando la matriz correspondiente por la matriz de coordenadas homogéneas de la figura. Posteriormente, a partir de las nuevas coordenadas, se reconstruye la figura transformada.

Visualizar transformaciones

A continuación se muestra la visualización del rectángulo original y del rectángulo transformado luego de aplicar la rotación y la traslación descritas anteriormente.

```
# 1. Definir un nuevo polígono (rectángulo)
rectangulo = Poligono([
    Punto(0.0, 0.0),
    Punto(2.0, 0.0),
    Punto(2.0, 1.0),
    Punto(0.0, 1.0)
])

# 2. Crear matrices individuales y matriz compuesta
# Rotación de 30 grados y traslación de (2,1)
angulo = pi/6
R = matriz_rotacion(angulo)
T = matriz_traslacion(2.0, 1.0)

# Composición: primero rotar, luego trasladar
M_compuesta = T * R

# 3. Aplicar la transformación
rect_transformado = transformar(rectangulo, M_compuesta)

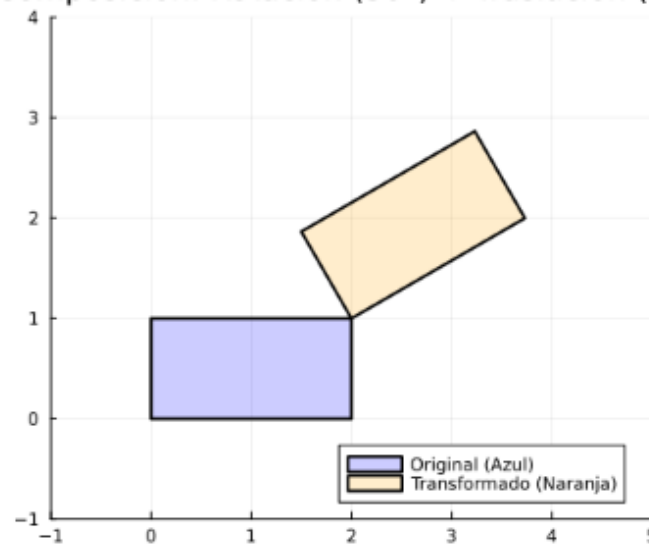
# 4. Crear el lienzo
plt_transformacion = plot(
    title="Composición: rotación (30 grados) y traslación (2,1)",
    aspect_ratio=:equal,
    legend=:bottomright,
    xlims=(-1, 5),
    ylims=(-1, 4)
)

# 5. Dibujar ambas figuras
dibujar(plt_transformacion, rectangulo,
        label="Original", color=:blue, lw=2)

dibujar(plt_transformacion, rect_transformado,
        label="Transformado", color=:orange, lw=2)

# 6. Mostrar resultado
display(plt_transformacion)
```

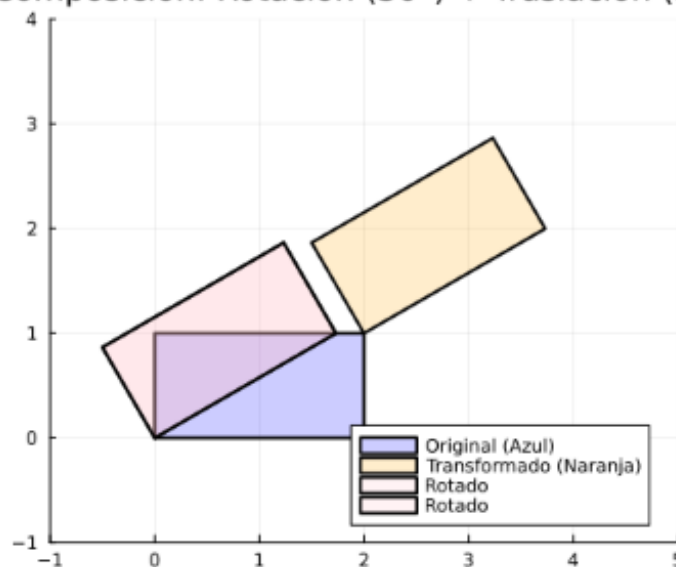
Composición: Rotación (30°) + Traslación (2,1)



De esta manera podemos visualizar el rectángulo luego de aplicar únicamente la rotación, antes de realizar la traslación.

```
R = matriz_rotacion(pi/6)
rect_transformado = transformar(rectangulo,R)
dibujar(plt_transformacion,rect_transformado,label = "Rotado",
color =:pink ,lw=2)
```

Composición: Rotación (30°) + Traslación (2,1)

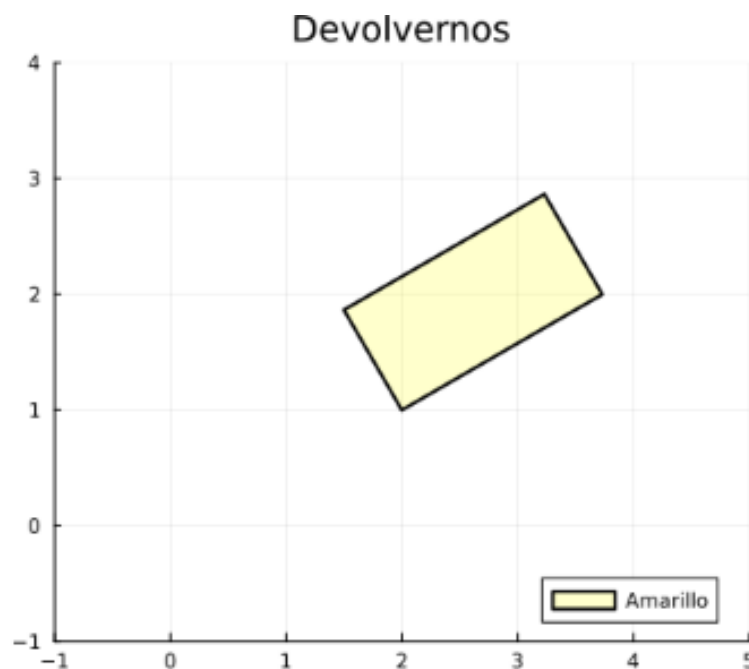


Deshaciendo una transformación

Ahora bien, supongamos que únicamente conocemos la figura transformada (en este caso el rectángulo amarillo) y queremos recuperar la figura original. Recordemos que dicha figura fue obtenida aplicando una rotación de 30° seguida de una traslación de $(2, 1)$.

Para deshacer la transformación debemos aplicar las operaciones inversas en orden contrario. Es decir, primero debemos deshacer la traslación y luego deshacer la rotación. En nuestro caso, la traslación inversa corresponde a trasladar $(-2, -1)$, mientras que la rotación inversa corresponde a una rotación de -30° .

```
rect_amarillo = transformar(rectangulo, M_compuesta)
# 4. Crear el lienzo
lienzo2 = plot(
    title="Devolvernos",
    aspect_ratio=:equal,
    legend=:bottomright,
    xlims=(-1, 5),
    ylims=(-1, 4)
)
dibujar(lienzo2, rect_amarillo, label="Amarillo", color=:yellow, lw=2)
```



Así tenemos en Julia el procedimiento para deshacer la transformación:

```
angulo = pi/6 # 30 grados

R = matriz_rotacion(angulo)
T = matriz_traslacion(2.0, 1.0)

T_inv = matriz_traslacion(-2.0, -1.0)
R_inv = matriz_rotacion(-angulo)

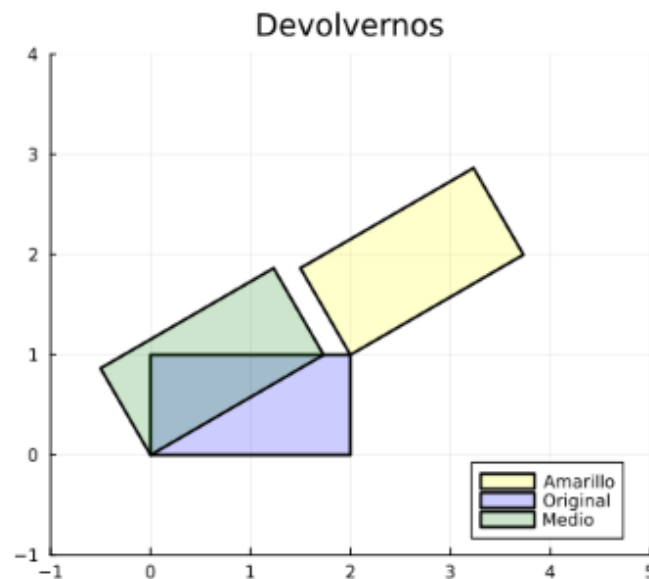
# transformación original
M_compuesta = T * R

# inversa
M_inversa = R_inv * T_inv

# figura transformada
rect_amarillo = transformar(rectangulo, M_compuesta)

# deshacer primero la traslación
rect_medio = transformar(rect_amarillo, T_inv)
dibujar(lienzo2, rect_medio, label="Medio", color=:green, lw=2)

# luego deshacer la rotación
rect_original = transformar(rect_medio, R_inv)
dibujar(lienzo2, rect_original, label="Original", color=:blue, lw=2)
```



4.5. Interpolación

Las notas de esta sección, así como la notación utilizada, sigue principalmente el desarrollo presentado en *Applied Linear Algebra* de Peter J. Olver y Chehrzad Shakiban [6].

El método de interpolación consiste en construir una función que represente un conjunto de datos discretos a partir de mediciones o valores observados. La función interpolante reproduce exactamente dichos datos.

De manera más formal, supongamos que se dispone de un conjunto de datos de la forma

$$(t_1, y_1), (t_2, y_2), \dots, (t_m, y_m),$$

donde los valores t_i son distintos. El problema de interpolación consiste en encontrar una función $y(t)$ tal que

$$y(t_i) = y_i, \quad i = 1, 2, \dots, m.$$

En el caso particular de la interpolación polinómica, se tiene el siguiente resultado fundamental: si se dispone de m puntos de datos distintos, entonces existe un único polinomio de grado a lo sumo $m - 1$ que pasa exactamente por todos ellos. Este polinomio se denomina **polinomio interpolante** y proporciona una función que describe el comportamiento de los datos dentro del intervalo considerado.

4.5.1. Interpolación mediante la matriz de Vandermonde

Sea un conjunto de puntos de datos

$$(t_1, y_1), (t_2, y_2), \dots, (t_m, y_m),$$

donde los valores t_i son distintos. Buscamos un polinomio de grado a lo sumo $m - 1$ de la forma

$$y(t) = \alpha_0 + \alpha_1 t + \alpha_2 t^2 + \dots + \alpha_{m-1} t^{m-1}$$

tal que

$$y(t_i) = y_i, \quad i = 1, \dots, m.$$

Al sustituir cada punto en el polinomio se obtiene el sistema lineal

$$\begin{aligned} \alpha_0 + \alpha_1 t_1 + \alpha_2 t_1^2 + \dots + \alpha_{m-1} t_1^{m-1} &= y_1 \\ \alpha_0 + \alpha_1 t_2 + \alpha_2 t_2^2 + \dots + \alpha_{m-1} t_2^{m-1} &= y_2 \\ &\vdots \\ \alpha_0 + \alpha_1 t_m + \alpha_2 t_m^2 + \dots + \alpha_{m-1} t_m^{m-1} &= y_m. \end{aligned}$$

Este sistema puede escribirse en forma matricial como

$$Ax = y,$$

donde

$$A = \begin{bmatrix} 1 & t_1 & t_1^2 & \dots & t_1^{m-1} \\ 1 & t_2 & t_2^2 & \dots & t_2^{m-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & t_m & t_m^2 & \dots & t_m^{m-1} \end{bmatrix}, \quad x = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{m-1} \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

La matriz A se conoce como la **matriz de Vandermonde**. Si los valores t_i son distintos, esta matriz es invertible y el sistema tiene una única solución, lo que garantiza la existencia y unicidad del polinomio interpolante.

4.5.2. Interpolación de Lagrange

Una alternativa al enfoque matricial mediante la matriz de Vandermonde consiste en construir directamente el polinomio interpolante mediante la fórmula de Lagrange. Este método proporciona una expresión explícita del polinomio que interpola los datos sin necesidad de resolver un sistema lineal.

Sea un conjunto de puntos de datos

$$(t_1, y_1), (t_2, y_2), \dots, (t_m, y_m),$$

donde los valores t_i son distintos. Se definen los **polinomios base de Lagrange** como

$$L_k(t) = \prod_{\substack{j=1 \\ j \neq k}}^m \frac{t - t_j}{t_k - t_j}, \quad k = 1, 2, \dots, m.$$

Estos polinomios poseen la propiedad fundamental

$$L_k(t_i) = \begin{cases} 1, & i = k, \\ 0, & i \neq k. \end{cases}$$

Utilizando estos polinomios base, el polinomio interpolante puede escribirse como

$$p(t) = \sum_{k=1}^m y_k L_k(t).$$

Por construcción, este polinomio satisface

$$p(t_i) = y_i, \quad i = 1, 2, \dots, m,$$

por lo que interpola exactamente todos los datos dados.

4.5.3. Interpolación a trozos

Aunque siempre es posible interpolar m puntos mediante un polinomio de grado $m - 1$, el uso de polinomios de grado elevado puede producir oscilaciones indeseadas, especialmente cerca de los extremos del intervalo de interpolación. Por esta razón, en muchas aplicaciones numéricas se prefiere utilizar **interpolación a trozos**.

La idea consiste en dividir el intervalo de interpolación en varios subintervalos y construir polinomios de bajo grado en cada uno de ellos. De esta manera, en lugar de utilizar un único polinomio global, se emplea una función definida por partes.

Supongamos que los puntos de datos satisfacen

$$t_1 < t_2 < \dots < t_m.$$

Se construyen entonces funciones polinómicas

$$p_1(t), p_2(t), \dots, p_{m-1}(t),$$

definidas respectivamente en los intervalos

$$[t_1, t_2], [t_2, t_3], \dots, [t_{m-1}, t_m].$$

La función interpolante se define entonces como

$$f(t) = \begin{cases} p_1(t), & t_1 \leq t \leq t_2, \\ p_2(t), & t_2 \leq t \leq t_3, \\ \vdots & \\ p_{m-1}(t), & t_{m-1} \leq t \leq t_m. \end{cases}$$

Este enfoque permite obtener aproximaciones más estables y suaves, especialmente cuando el número de datos es grande. Un caso particularmente importante de interpolación a trozos es el uso de **splines cúbicos**, donde cada tramo se aproxima mediante un polinomio de grado tres y se imponen condiciones de continuidad en los puntos de unión.

Ejemplos de implementación en Julia

1. Interpolación mediante la matriz de Vandermonde

Consideremos los puntos

$$(0, 1), \quad (1, 3), \quad (2, 2).$$

Buscamos el polinomio interpolante de grado a lo sumo 2.

```
using LinearAlgebra

# datos
t = [0.0, 1.0, 2.0]
y = [1.0, 3.0, 2.0]

# matriz de Vandermonde
A = [ti^j for ti in t, j in 0:2]

# coeficientes del polinomio interpolante
x = A \ y

println("Coeficientes del polinomio:")
println(x)
-----
Coeficientes del polinomio:
[1.0, 3.5, -1.5]
```

Si el vector obtenido es

$$x = \begin{bmatrix} 1 \\ 3,5 \\ -1,5 \end{bmatrix},$$

entonces el polinomio interpolante es

$$p(t) = 1 + 3,5t - 1,5t^2.$$

Evaluación del polinomio interpolante

Una vez obtenidos los coeficientes, el polinomio puede evaluarse en cualquier punto del intervalo.

```
# función polinómica
p(t) = x[1] + x[2]*t + x[3]*t^2

println("p(0) = ", p(0.0))
println("p(1) = ", p(1.0))
println("p(2) = ", p(2.0))
println("p(1.5) = ", p(1.5))
-----
p(0) = 1.0
p(1) = 3.0
p(2) = 2.0
p(1.5) = 2.875
```

Los valores $p(0)$, $p(1)$ y $p(2)$ deben coincidir exactamente con los datos dados, mientras que $p(1,5)$ proporciona un valor intermedio estimado por interpolación.

2. Interpolación de Lagrange

El mismo polinomio interpolante puede construirse mediante la fórmula de Lagrange. Para los datos anteriores, los polinomios base son

$$L_k(t) = \prod_{\substack{j=1 \\ j \neq k}}^m \frac{t - t_j}{t_k - t_j}, \quad k = 1, 2, 3.$$

En Julia, una implementación sencilla es la siguiente:

```
# función de interpolación de Lagrange
function lagrange_interp(tdata, ydata, t)
    n = length(tdata)
    s = 0.0
    for k in 1:n
        L = 1.0
        for j in 1:n
            if j != k
                L *= (t - tdata[j]) / (tdata[k] - tdata[j])
            end
        end
        s += ydata[k] * L
    end
    return s
end

println("p(1.5) = ", lagrange_interp(t, y, 1.5))
-----
p(1.5) = 2.875
```

Este procedimiento construye directamente el polinomio interpolante sin resolver explícitamente el sistema lineal asociado a la matriz de Vandermonde.

3. Interpolación a trozos

Cuando el número de puntos es grande, puede ser preferible utilizar interpolación a trozos. En el caso más simple, se puede interpolar de manera lineal entre pares consecutivos de puntos.

Consideremos los datos

$$(0, 1), \quad (1, 3), \quad (2, 2), \quad (3, 4).$$

La interpolación lineal a trozos consiste en usar una recta distinta en cada subintervalo.

```
# datos
t = [0.0, 1.0, 2.0, 3.0]
y = [1.0, 3.0, 2.0, 4.0]

# interpolación lineal a trozos
function piecewise_linear(tdata, ydata, x)
    n = length(tdata)
    for i in 1:n-1
        if tdata[i] <= x <= tdata[i+1]
            m = (ydata[i+1] - ydata[i]) / (tdata[i+1] - tdata[i])
            return ydata[i] + m*(x - tdata[i])
        end
    end
    error("El punto está fuera del intervalo de interpolación.")
end

println("f(0.5) = ", piecewise_linear(t, y, 0.5))
println("f(1.5) = ", piecewise_linear(t, y, 1.5))
println("f(2.5) = ", piecewise_linear(t, y, 2.5))

-----
f(0.5) = 2.0
f(1.5) = 2.5
f(2.5) = 3.0
```

En este caso no se utiliza un único polinomio global, sino una función definida por tramos. Esta idea constituye la base de métodos más avanzados, como los splines cúbicos.

Capítulo 5

Transformaciones y Matrices

Se seguirá principalmente el tratamiento desarrollado en *Introduction to Applied Linear Algebra* de Boyd y Vandenberghe [5].

Recordemos que una **transformación** es una función que toma un vector como entrada y produce otro vector como salida.

Si T es una transformación entre espacios vectoriales, escribimos

$$T : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

lo cual significa que T toma un vector $x \in \mathbb{R}^n$ y lo transforma en un vector $y \in \mathbb{R}^m$.

5.1. Transformaciones lineales

Una transformación se dice **lineal** si satisface las propiedades de aditividad y homogeneidad:

$$T(x_1 + x_2) = T(x_1) + T(x_2),$$

$$T(\lambda x) = \lambda T(x).$$

Estas propiedades garantizan que la transformación preserve la estructura lineal del espacio vectorial.

5.1.1. Representación mediante matrices

Toda transformación lineal puede representarse mediante una matriz. Si $A \in \mathbb{R}^{m \times n}$, la transformación asociada se escribe como

$$T(x) = Ax.$$

Por lo tanto, aplicar una transformación lineal equivale a realizar una multiplicación matriz–vector.

5.1.2. Composición de transformaciones

Si T_1 y T_2 son transformaciones lineales, su composición se define por

$$(T_2 \circ T_1)(x) = T_2(T_1(x)).$$

Si

$$T_1(x) = A_1x, \quad T_2(x) = A_2x,$$

con

$$A_1 \in \mathbb{R}^{p \times n}, \quad A_2 \in \mathbb{R}^{m \times p},$$

entonces

$$(T_2 \circ T_1)(x) = A_2A_1x.$$

Note que el orden de multiplicación es importante: primero se aplica T_1 y luego T_2 . Por tanto, la matriz asociada a la transformación compuesta es el producto A_2A_1 .

5.1.3. Transformación inversa

Una transformación lineal es **invertible** si existe una transformación T^{-1} tal que

$$T^{-1}(T(x)) = x.$$

Si $T(x) = Ax$ con A cuadrada, entonces la transformación inversa está representada por la matriz A^{-1} que satisface

$$AA^{-1} = A^{-1}A = I,$$

donde I es la matriz identidad. La inversa existe únicamente si A es **no singular**, es decir, si $\det(A) \neq 0$.

5.2. Matrices Invertibles

5.2.1. Definición y caracterizaciones equivalentes

Sea $A \in \mathbb{R}^{n \times n}$ una matriz cuadrada.

Decimos que A es **invertible** si existe una matriz $A^{-1} \in \mathbb{R}^{n \times n}$ tal que

$$AA^{-1} = A^{-1}A = I,$$

donde I es la matriz identidad de tamaño $n \times n$.

La matriz A^{-1} se denomina **inversa** de A . Una matriz que posee inversa se dice **invertible** o **no singular**. Una matriz cuadrada que no posee inversa se denomina **singular**. Si una matriz tiene inversa, esta es **única**.

5.2.2. Teorema de la matriz invertible

Para una matriz cuadrada $A \in \mathbb{R}^{n \times n}$, las siguientes condiciones son equivalentes:

- A es invertible.
- Las columnas de A son linealmente independientes.
- Las filas de A son linealmente independientes.
- A tiene una inversa izquierda.
- A tiene una inversa derecha.

5.2.3. Relación con sistemas lineales

Consideremos el sistema lineal

$$Ax = b,$$

donde $A \in \mathbb{R}^{n \times n}$ y $b \in \mathbb{R}^n$.

Si la matriz A es invertible, entonces el sistema tiene una **única solución**, dada por

$$x = A^{-1}b.$$

Esto se verifica fácilmente observando que

$$A(A^{-1}b) = (AA^{-1})b = Ib = b.$$

Por lo tanto, cuando A es invertible, el sistema lineal posee una solución única para cualquier vector b .

5.2.4. Inversa de matrices triangulares

Una matriz triangular superior tiene la forma

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ 0 & 0 & \ddots & u_{3n} \\ 0 & 0 & \cdots & u_{nn} \end{bmatrix}.$$

Una matriz triangular es invertible si y solo si todos los elementos de su diagonal son distintos de cero.

Además, la inversa de una matriz triangular es también triangular.

5.2.5. Inversa de matrices ortogonales

Una matriz $Q \in \mathbb{R}^{n \times n}$ se denomina **ortogonal** si satisface

$$Q^T Q = I.$$

Una propiedad fundamental de las matrices ortogonales es que su inversa coincide con su traspuesta:

$$Q^{-1} = Q^T.$$

Esto implica que las transformaciones ortogonales preservan la norma y los ángulos entre vectores.

Ejemplos Matrices Invertibles

Ejemplo 1: verificar si la matriz es invertible

Consideremos la matriz

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}.$$

Su inversa es

$$A^{-1} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}.$$

Podemos verificar que

$$AA^{-1} = A^{-1}A = I.$$

Por lo tanto, la matriz A es invertible.

En Julia también podemos verificar este resultado:

```
using LinearAlgebra
A = [2 1; 1 1]

display(inv(A))

display(A*inv(A))

-----
2x2 Matrix{Float64}:
 1.0 -1.0
-1.0  2.0
2x2 Matrix{Float64}:
 1.0  0.0
 0.0  1.0
```

Note que el producto AA^{-1} produce la matriz I de dimensión 2×2 y por tanto esto confirma que la matriz A es invertible.

Ejemplo 2: teorema de la matriz invertible

Sea

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

Las columnas de A son

$$a_1 = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad a_2 = \begin{bmatrix} 2 \\ 4 \end{bmatrix}.$$

Estas columnas son linealmente independientes, ya que el determinante de la matriz es

$$\det(A) = -2 \neq 0.$$

Por lo tanto, la matriz A es invertible.

De manera análoga en Julia:

```
using LinearAlgebra

A = [1 2; 3 4]
println(det(A))
display(inv(A))
println(inv(A))
-----
-2.0
2x2 Matrix{Float64}:
-2.0  1.0
 1.5 -0.5
[-1.9999999999999996  0.9999999999999998; 1.4999999999999998 -0.4999999999999999]
```

Notas:

En Julia, cuando una expresión se escribe directamente, el entorno de ejecución muestra matrices y vectores en un formato estructurado. Sin embargo, funciones como `println` imprimen los resultados como una representación textual en una sola línea.

Además, los valores numéricos se muestran con su representación en punto flotante, por lo que pueden aparecer con muchos decimales y no exactamente en forma aproximada. Por esta razón utilizamos la función `display()` para forzar la visualización en formato matricial, que resulta más clara para interpretar los resultados.

Ejemplo 3: solución sistema lineal

Consideremos el sistema lineal

$$Ax = b,$$

donde

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Como la matriz A es invertible, la solución del sistema es

$$x = A^{-1}b.$$

Con Julia podemos calcular la solución del sistema lineal $Ax = b$ de dos maneras.

Método 1

Calcular explícitamente la inversa de la matriz y luego multiplicarla por el vector b :

$$x = A^{-1}b.$$

En Julia esto se realiza mediante la expresión `inv(A)*b`.

Método 2

Resolver el sistema lineal directamente sin calcular la inversa de la matriz. Esto se realiza mediante el operador `\`:

`A\b`.

Este método utiliza algoritmos de factorización interna y **suele ser más estable numéricamente y más eficiente** que calcular explícitamente la inversa de la matriz.

Visualmente tenemos en Julia:

```
using LinearAlgebra

A = [2 1; 1 3]
b = [1, 2]

display(inv(A)*b)
display(A\b)
-----
2-element Vector{Float64}:
 0.19999999999999996
 0.6000000000000001
2-element Vector{Float64}:
 0.2
 0.6
```

Bibliografía

- [1] Jeff Bezanson, Stefan Karpinski, Viral B. Shah, and Alan Edelman. Julia: A fast dynamic language for technical computing. *arXiv preprint arXiv:1209.5145*, 2012.
- [2] Julia Language. Julia 1.10 documentation. <https://docs.julialang.org/en/v1/>, 2024.
- [3] Lloyd N. Trefethen and David Bau III. *Numerical Linear Algebra*. SIAM, 1997.
- [4] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4 edition, 2013.
- [5] Stephen Boyd and Lieven Vandenberghe. *Introduction to Applied Linear Algebra: Vectors, Matrices, and Least Squares*. Cambridge University Press, 2018.
- [6] Peter J. Olver and Chehrzad Shakiban. *Applied Linear Algebra*. Springer, 2 edition, 2018.